Go for Distributed Systems

25-Day Roadmap to Build Real Infra Muscle

Target	Output	Rules
Service discovery, watches, leases, RPC, concurrency	A toy discovery system with client cache and failure handling	No CRUD fluff. Build, break, test, explain.

Designed for an engineer who already knows the basics and wants the systems path, fast.

How to use this roadmap

This plan assumes you already know enough Go syntax to write small programs. The mission here is to become good at networked, concurrent, failure-aware Go.

Every day has one main output. Protect that. Do not replace implementation with passive reading.

Your north star by Day 25: a toy service-discovery system in Go with registration, leases, watch streams, a client-side cache, clean shutdown, tests, and failure handling.

Operating rules

- Code before you optimize your reading list.
- Every networked component must have timeouts, cancellation, and graceful shutdown.
- Every background goroutine must have a clear lifecycle.
- Run tests with the race detector when concurrency enters the picture.
- Write short notes after each day: what broke, what confused you, what became clear.

What you will build

- A timeout-aware TCP or HTTP service with proper lifecycle management.
- A gRPC registry API with register, heartbeat, resolve, and watch operations.
- An in-memory revisioned registry with TTL leases and event delivery.
- A client library that keeps a local cache fresh via watch streams and falls back to resync when needed.

Week 1 - Go as a systems language

Goal: become sharp at context, concurrency, lifecycle, and failure-aware networking.

Day	Focus	Build / Drill	Reading / Reference	Deliverable
1	Context and lifecycle	Write a tiny HTTP service with request-scoped context, timeouts, and graceful shutdown.	Read context and net/http docs. Trace cancellation through one request path.	Clean shutdown demo
2	Goroutines and select	Build a worker pool with context cancellation and bounded queues.	Review sync vs channels trade-offs. Focus on leak patterns.	Pool with tests
3	Mutexes, atomics, race thinking	Add shared state safely: counters, maps, and lock-protected stats.	Read sync and atomic docs. Run race detector.	Race-free state
4	Retries, deadlines, jitter	Build a client wrapper that retries only retryable failures and	Study timeout/error handling patterns in net/http.	Retrying client

Day	Focus	Build / Drill	Reading / Reference	Deliverable
		respects context.		
5	Structured logs and health endpoints	Add request IDs, readiness/liveness, and observable shutdown logs.	Read a little about failure budgets and health semantics.	Operationalized service
6	Testing concurrent code	Write table-driven tests for shutdown, timeout, and backpressure behavior.	Use go test -race and benchmark one hot path.	Solid test file
7	Checkpoint day	Refactor the week into a tidy mini-service template you can reuse.	Skim your own code and delete fluff.	Reusable skeleton

Week 2 - Service boundaries and discovery primitives

Goal: cross the line from “Go service” to “Go control-plane component.”

Day	Focus	Build / Drill	Reading / Reference	Deliverable
8	gRPC basics	Define a proto for a registry service: Register, Heartbeat, Resolve, Watch.	Read gRPC Go quickstart and understand generated code shape.	Proto + server stub
9	Unary RPC discipline	Implement Register and Resolve. Validate inputs and return useful errors.	Review status codes and deadline propagation.	Working unary API
10	Streaming RPC	Implement Watch as a server stream that emits instance changes.	Read about stream lifecycles and cancellation.	Watch stream v1
11	In-memory registry model	Create service, instance, and revision structures. Keep writes serialized.	Read about revisioned state and event streams.	Registry core
12	Lease manager	Add TTL leases and keepalive handling. Expire instances in background loops.	Study leases/TTL semantics in etcd-style systems.	Lease expiry path
13	Client library	Write a Go client that registers itself,	Read your own server API like a user would.	Thin client lib

Day	Focus	Build / Drill	Reading / Reference	Deliverable
		heartbeats, and resolves peers.		
14	Checkpoint day	Run a two-process demo: one server, two instances, one resolver client.	Note every rough edge before week 3.	End-to-end demo

Week 3 - Watches, caches, and recovery

Goal: build the parts that make service discovery feel real instead of toy-like.

Day	Focus	Build / Drill	Reading / Reference	Deliverable
15	Revisioned watchable state	Emit create/update/delete events with monotonically increasing revisions.	Read about watch streams, missed events, and resync patterns.	Event model
16	Slow consumers and buffers	Handle blocked watchers without freezing the whole registry.	Study backpressure choices: drop, block, disconnect, or snapshot.	Watcher policy
17	Resync after failure	Teach the client to re-list state and resume after a broken watch.	Read about compaction conceptually even if your toy system is simpler.	Recovery path
18	Local cache + resolver	Maintain a concurrency-safe client cache used by Resolve without round-tripping each time.	Think through freshness vs availability.	Cached resolver
19	Failure drills	Kill heartbeats, delay watchers, restart client, and observe convergence.	Write down the failure semantics you actually see.	Failure notes
20	Testing ugly paths	Add tests for lease expiry, duplicate IDs, watch interruption, and stale cache recovery.	Hammer go test -race until clean.	Concurrency test pack
21	Checkpoint day	Refactor event loops and package	Make your shutdown path boring and	Cleaner architecture

Day	Focus	Build / Drill	Reading / Reference	Deliverable
		boundaries based on what failed under stress.	reliable.	

Week 4 - Polish, persistence, and engineering taste

Goal: turn the project from “works on my laptop” into “I understand what I built.”

Day	Focus	Build / Drill	Reading / Reference	Deliverable
22	Operational visibility	Add metrics or lightweight stats for leases, watchers, revisions, dropped events, and cache age.	Read a bit on pprof or simple benchmarking.	Inspectable system
23	Snapshot and restore	Serialize registry state and reload it cleanly on restart.	Think about what state is authoritative vs reconstructable.	Basic persistence
24	Design write-up	Write a 1-2 page design note: API, data model, loops, failure modes, trade-offs.	Use precise language. No hype.	Design doc
25	Final gauntlet	Run the full demo, record what happens under failure, and clean the repo for future you.	Skim Raft/etcd notes again and connect them to your implementation.	Portfolio-grade project

What good looks like by Day 25

- Your registry server accepts registrations, tracks TTL leases, expires dead instances, and publishes change events.
- Your watch stream can be interrupted and your client can recover by re-listing and resubscribing.
- Your client-side cache serves resolves without hammering the server and still converges back to truth.
- Every major loop is context-aware and shuts down cleanly.
- You have tests for lease expiry, duplicate registration, watch recovery, shutdown, and concurrent access.

Do not waste your 25 days on this

- No ORM tutorials, no generic REST todo app, no “clean architecture” cosplay.

- Do not bike-shed about package layout before you have a working event loop.
- Do not read five papers in a row without implementing one idea from them.
- Do not hide concurrency bugs behind retries and prayer.

Reading spine

- Go standard library docs: context, net/http, sync, sync/atomic, runtime, testing.
- gRPC Go basics once you start service boundaries.
- etcd docs around leases, watches, revisions, compaction, and client behavior.
- The Raft paper as background for why control-plane systems care about leader/quorum semantics.

Daily cadence

90-120 min	Build the day's main artifact before touching extra reading.
20 min	Read the minimum theory needed to sharpen the artifact.
20-30 min	Test, refactor lightly, and write 3-5 bullet notes.
5 min	Commit with a message that says what changed and what still feels wrong.

Last note

This roadmap is not trying to make you "good at Go" in the abstract. It is trying to make you useful in the exact lane where Go shines: control planes, infra daemons, service discovery, watch-driven state, and systems that keep functioning while the world gets messy.